



Universidad Autónoma de Nuevo
León

Facultad de Ingeniería Mecánica y Eléctrica

Práctica 3

Diagramas de Minería de Procesos

Nombre	Ranfery Josua Peregrina Morales
Matricula	1924910
Grupo	507
Carrera	ITS
Asignatura	Tópicos Selectos de Ciencias de la Ing. III
Docente	Raymundo Said Zamora Pequeño
Fecha	22/3/26

Periodo Enero – Junio 2026

Contenido

Objetivo.....	3
Marco teórico.....	3
Minería de procesos y su relación con la ciencia de datos	3
Registros de eventos y estructura de los procesos.....	3
Descubrimiento de procesos (Process Discovery)	4
Grafos de seguimiento directo (Directly-Follows Graphs)	4
Sistemas de transición (Transition Systems).....	4
Redes de Petri	5
Modelado de secuencia, decisión y concurrencia	5
Secuencia.....	5
Elección (XOR-split)	6
Paralelismo (AND-split)	6
Workflow nets (WF-nets)	6
Herramientas de minería de procesos	6
Desarrollo	7
Diagrama de disco representado en diferentes tipos de diagrama.....	7
Sistemas de Transición (Transition Systems)	7
Mapa de procesos de eventdataR.....	8
Cálculos y resultados	11
Cálculos del mapa de procesos en R	11
Cálculos de actividades y frecuencias	11
Cálculos del mapa de procesos	12
Síntesis del análisis del ejercicio con edeaR.....	13
Conclusión de la práctica	14
Referencias.....	14

Objetivo

Analizar un proceso mediante su representación en diferentes tipos de diagramas

Marco teórico

Minería de procesos y su relación con la ciencia de datos

La minería de procesos es una disciplina emergente que se sitúa en la intersección entre la minería de datos, la gestión de procesos de negocio y la informática. Su objetivo principal es analizar los registros de eventos generados por sistemas de información para descubrir, monitorear y mejorar procesos reales [1].

A diferencia de los enfoques tradicionales de análisis organizacional, que dependen de entrevistas o documentación manual, la minería de procesos se basa en datos objetivos registrados automáticamente, lo que permite obtener representaciones precisas del comportamiento real de los procesos [1]. Esto la convierte en una herramienta clave para la transformación digital y la mejora continua en organizaciones modernas.

Los registros de eventos (event logs) constituyen la fuente principal de información en minería de procesos. Cada evento contiene típicamente:

- Identificador de caso (case ID)
- Actividad ejecutada
- Marca temporal (timestamp)

Estos elementos permiten reconstruir la secuencia de actividades que forman cada instancia de un proceso [1].

Registros de eventos y estructura de los procesos

Un proceso de negocio puede definirse como un conjunto de actividades interrelacionadas que transforman entradas en salidas para alcanzar un objetivo organizacional [2]. Cuando estos procesos son ejecutados en sistemas informáticos, cada actividad genera un evento que puede ser almacenado en un log.

Formalmente, un registro de eventos se define como una colección de trazas, donde cada traza representa la secuencia de actividades ejecutadas en un caso específico [1]. Esto puede expresarse como:

$$L = \{\sigma_1, \sigma_2, \sigma_3 \dots\}$$

Donde σ_i es la secuencia ordenada de eventos.

El análisis de estos registros permite identificar:

- el orden real de las actividades
- la frecuencia de ejecución
- variaciones y excepciones del proceso

Descubrimiento de procesos (Process Discovery)

El descubrimiento de procesos es una de las tareas fundamentales de la minería de procesos. Consiste en generar automáticamente un modelo del proceso a partir de los registros de eventos, sin utilizar información previa del proceso [1].

Entre los algoritmos más conocidos se encuentran:

- Alpha Miner
- Heuristic Miner
- Inductive Miner

Estos algoritmos analizan las relaciones de precedencia entre actividades para reconstruir el flujo de trabajo subyacente [3].

El descubrimiento de procesos permite obtener modelos en diferentes formalismos, entre ellos:

- Diagramas de flujo
- Sistemas de transición
- Redes de Petri
- Redes de flujo de trabajo
- Modelos YAWL
- Cadenas de procesos impulsadas por evento
- Redes causales
- Árboles de proceso

Grafos de seguimiento directo (Directly-Follows Graphs)

Uno de los modelos intermedios más utilizados es **el grafo de seguimiento directo** (Directly-Follows Graph, DFG). En este tipo de grafo, los nodos representan actividades y las aristas representan que una actividad es seguida directamente por otra en al menos una traza del registro [1].

Los DFG son útiles para:

- visualizar la estructura general del proceso
- detectar cuellos de botella
- identificar rutas alternativas y excepciones

Sin embargo, estos grafos no son modelos formales ejecutables, por lo que se utilizan principalmente como una representación exploratoria [4].

Sistemas de transición (Transition Systems)

Un sistema de transición es un modelo formal que describe el comportamiento de un proceso en términos de estados y transiciones entre dichos estados. En este modelo, cada estado representa una situación del proceso y cada transición representa la ejecución de una actividad [1].

Formalmente, un sistema de transición se define como una tupla:

$$T = (S, E, T)$$

Donde

- S es el conjunto de estados
- E es el conjunto de eventos o actividades
- $T \subseteq S * E * S$ es el conjunto de transiciones

Este modelo permite representar el comportamiento dinámico de un proceso y analizar propiedades como:

- alcanzabilidad
- determinismo
- comportamiento repetitivo

Los sistemas de transición son especialmente útiles para comprender la lógica secuencial de los procesos antes de convertirlos en modelos más expresivos como las redes de Petri.

Redes de Petri

Las redes de Petri son uno de los formalismos más importantes para modelar procesos concurrentes y distribuidos. Fueron introducidas por Carl Adam Petri en la década de 1960 y se han convertido en un estándar para el análisis formal de sistemas de flujo de trabajo [5].

Una red de Petri se define como una tupla:

$$N=(P,T,F)$$

donde:

- P es el conjunto de lugares (places)
- T es el conjunto de transiciones
- F es el conjunto de arcos dirigidos [5]

En este modelo:

- los **lugares** representan estados o condiciones
- las **transiciones** representan actividades o eventos
- los **tokens** representan la ejecución del proceso

La evolución de una red de Petri se produce mediante el disparo de transiciones, lo que permite modelar formalmente la dinámica del proceso [5].

Modelado de secuencia, decisión y concurrencia

Una de las principales ventajas de las redes de Petri es su capacidad para representar diferentes patrones de control de flujo, como:

Secuencia

La secuencia ocurre cuando una actividad debe completarse antes de que otra pueda comenzar. Este patrón se representa mediante una cadena de lugares y transiciones conectados linealmente [5].

Elección (XOR-split)

La elección ocurre cuando el proceso puede seguir uno de varios caminos posibles. En una red de Petri, esto se modela mediante un lugar con múltiples transiciones salientes, de las cuales solo una puede dispararse [5].

Paralelismo (AND-split)

El paralelismo permite que múltiples actividades se ejecuten simultáneamente. En las redes de Petri, esto se representa mediante una transición que produce tokens en múltiples lugares, habilitando varias actividades concurrentes [5].

Estos patrones permiten modelar procesos complejos de manera precisa y ejecutable.

Workflow nets (WF-nets)

Una Workflow net es una clase especial de red de Petri diseñada específicamente para modelar procesos de negocio. Una WF-net debe cumplir tres propiedades:

1. Tener un único lugar inicial
2. Tener un único lugar final
3. Todos los nodos deben ser alcanzables desde el inicio y conducir al final [1]

Estas restricciones garantizan que el modelo represente un proceso bien estructurado, sin partes desconectadas ni ejecuciones incompletas.

Herramientas de minería de procesos

Existen diversas herramientas que implementan algoritmos de minería de procesos. Entre las más utilizadas se encuentran:

- ProM Framework
- Disco
- Celonis

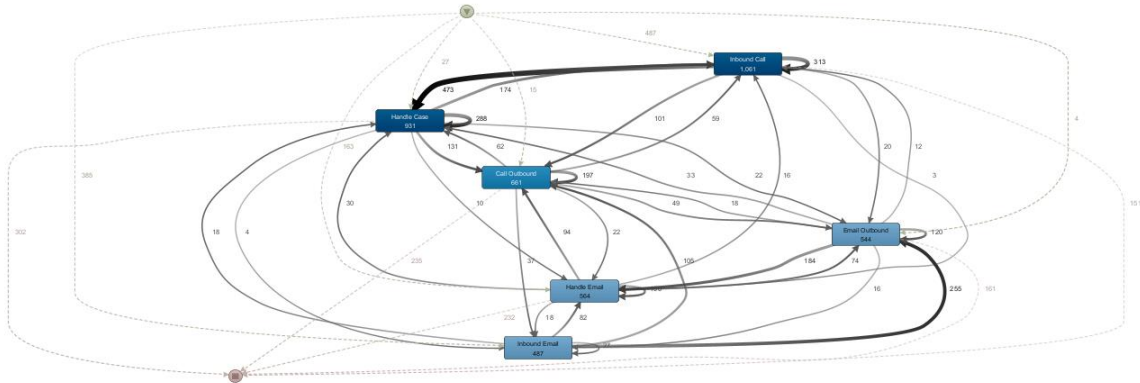
Estas herramientas permiten importar registros de eventos, descubrir modelos de proceso y realizar análisis de conformidad y desempeño [6].

En particular, Disco es una herramienta comercial que facilita la visualización de DFG y la exploración interactiva de los procesos, lo que la hace adecuada para actividades de laboratorio y aprendizaje [6].

Desarrollo

Diagrama de disco representado en diferentes tipos de diagrama

Según el diagrama obtenido de la *Práctica 2* en donde basándonos en un registro de procesos predeterminado en el *Software Disco*[2], se hizo un análisis del mapa de procesos de todos los procesos que demostraron atención al cliente. El mapa está a continuación.



El mapa de procesos ilustra la frecuencia de ruta en procesos de todos los casos seleccionados. Y para esta práctica, vamos a hacer una representación de dicho mapa de procesos a partir de otros tipos de diagrama mencionados en el libro de Wil van der Aalst[6].

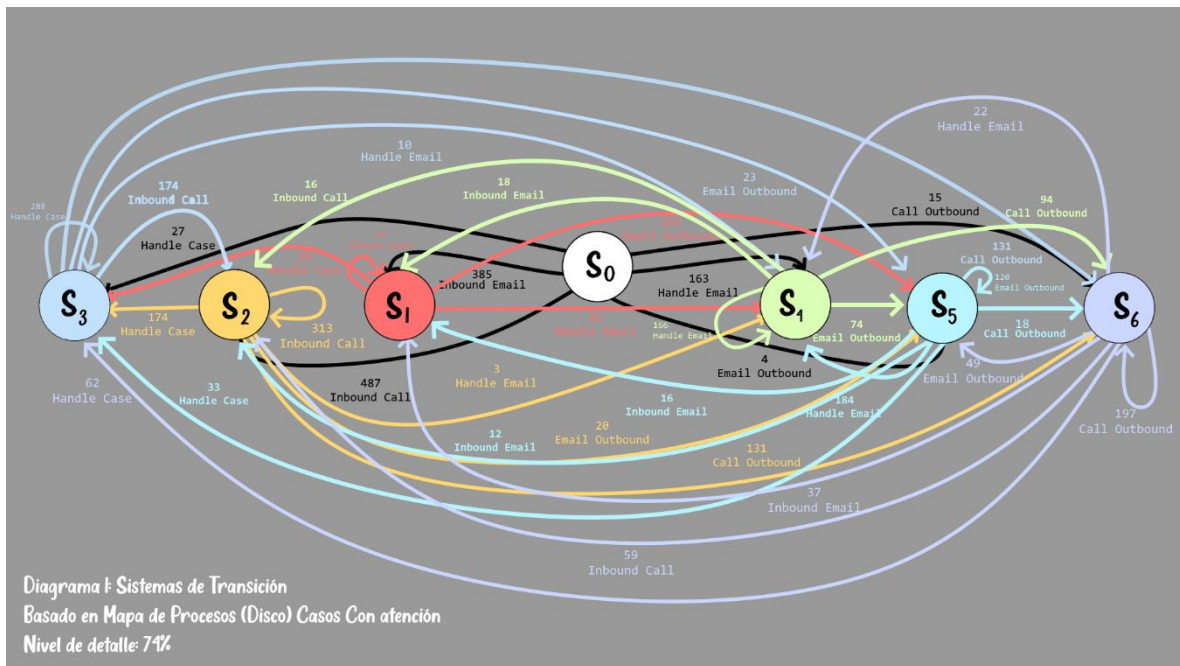
El primer gran paso para "traducir" nuestro diagrama es entender qué tienes ahora mismo. Lo que *Disco*[1] nos generó (y que filtramos al 74% de precisión) se conoce como un **Grafo de Sucesión Directa (Directly-Follows Graph o DFG)**. En un DFG, las cajas son nuestras actividades ("Llamada", "Correo") y las flechas simplemente dicen "esto pasó después de esto".

Los diagramas formales del libro cambian estas reglas. Así que para *traducir* a cada diagrama tenemos que leer e interpretar esas reglas. Y luego ilustrarlo siguiendo dichas reglas como se hizo en la sección de *Cálculos y resultados*.

Sistemas de Transición (Transition Systems)

Para traducir eso a un Sistema de Transición, tenemos que **invertir** esa lógica visual.

- **Las flechas** son ahora las actividades (Inbound Email, Handle Case, Call Outbound).
- **Los círculos (nodos)** son los "Estados". Un estado es simplemente un momento en el tiempo o una situación en la que está el caso, como una sala de espera antes de que ocurra la siguiente actividad. No tienen nombre de actividad, usualmente solo se enumeran $s_1, s_2, s_3...$



Mapa de procesos de eventdataR

Después de instalar las librerías de *bupaR*, *edeaR*, instalamos '*DiagrammeR*', '*plotly*', '*BH*', que no eran necesarias para la práctica en sí, pero que fueron requeridas para instalar *processmapR_0.5.7*.

Después tomamos el script de ejemplo presentado en la *Clase de laboratorio "bupa.txt"*

```
library(bupaR)
library(edeaR)
library(processmapR)
class(patients)
n_activities(patients)
mapping(patients)
dim(patients)
n_activities(patients)
activity_labels(patients)
activities(patients)
processing_time(patients, "activity", units="mins")
processing_time(patients, level="log", units="days")
patients %>% filter_activity_presence("Blood test") %>%
processing_time(level="log", units="hours")
activity_frequency(sepsis, level = "activity") %>% arrange(relative)
patients_subset<-filter_activity_presence(sepsis, "CRP")
patients_subset %>% process_map(type = frequency("absolute"))
```

En este script, el objeto principal es `patients`. Para cumplir con la tarea, debemos elegir uno de los datasets que aparecen en la página de *Ayuda del Package eventdataR*.

Nuestra elección es [traffic_fines](#)

Primeramente cargamos todas las librerías para trabajar.

```
> library(bupaR)
> library(edeaR)
> library(processmapR)
> library(eventdataR) #Esta para acceder a los datos con los que trabajamos:)
>
```

Después, del script original reemplazamos “patients” por “traffic_fines”. De donde obtenemos información sobre esta tabla con registros de datos. Tiene 11 actividades distintas, organizadas por id. Tienen registros de los recursos implicados (De los que hablaremos más tarde) e información sobre su tiempo de duración.

```
> # 1. Exploración inicial (reemplazamos patients por traffic_fines)
> class(traffic_fines)
[1] "eventlog" "log" "tbl_df" "tbl" "data.frame"
> n_activities(traffic_fines)
[1] 11
> mapping(traffic_fines)
Case identifier: case_id
Activity identifier: activity
Resource identifier: resource
Activity instance identifier: activity_instance_id
Timestamp: timestamp
Lifecycle transition: lifecycle
```

Después hacemos una exploración de las dimensiones del dataset. Y obtenemos información sobre las etiquetas del data set. Por lo descriptivos que son sus nombres y recordando que es un registro de multas de tráfico, entendemos que cada registro del *log* es una actividad de la multa de tráfico. Desde crearla hasta notificar al infractor o ir a juicio por apelación.

```
> # 2. Dimensiones y etiquetas
> dim(traffic_fines)
[1] 34724 18
> activity_labels(traffic_fines)
[1] Create Fine Send Fine Insert Fine Notification
[4] Add penalty Send for Credit Collection Payment
[7] Insert Date Appeal to Prefecture Send Appeal to Prefecture Receive Result Appeal from Prefecture
[10] Notify Result Appeal to Offender Appeal to Judge
11 Levels: Add penalty Appeal to Judge Create Fine Insert Date Appeal to Prefecture ... Send for Credit Collection
> activities(traffic_fines)
# A tibble: 11 x 3
  activity absolute_frequency relative_frequency
  <fct> <int> <dbl>
1 Create Fine 10000 0.288
2 Send Fine 6570 0.189
3 Payment 4910 0.141
4 Add penalty 4635 0.133
5 Insert Fine Notification 4635 0.133
6 Send for Credit Collection 3387 0.0975
7 Insert Date Appeal to Prefecture 232 0.00668
8 Send Appeal to Prefecture 227 0.00654
9 Receive Result Appeal from Prefecture 55 0.00158
10 Notify Result Appeal to offender 54 0.00156
11 Appeal to Judge 19 0.000547
>
```

Después analizamos los tiempos de procesamiento, es decir, lo que dura cada una de las actividades registradas. Pero aquí nos encontraremos con un problema:

```

> # 3. Tiempos de procesamiento
> processing_time(traffic_fines, "activity", units="mins")
# A tibble: 11 x 11
  activity      min      q1      mean      median      q3      max      st_dev      iqr      total      relative_frequency
  <fct>      <drtn>      <drtn>      <drtn>      <drtn>      <drtn>      <drtn>      <dbl>      <dbl>      <drtn>      <dbl>
1 Create Fine      Inf mins      NA mins      NaN mins      NA mins      NA mins      -Inf mins      NA      NA      0 mins      0.288
2 Send Fine      Inf mins      NA mins      NaN mins      NA mins      NA mins      -Inf mins      NA      NA      0 mins      0.189
3 Payment      Inf mins      NA mins      NaN mins      NA mins      NA mins      -Inf mins      NA      NA      0 mins      0.141
4 Add penalty      Inf mins      NA mins      NaN mins      NA mins      NA mins      -Inf mins      NA      NA      0 mins      0.133
5 Insert Fine Notification      Inf mins      NA mins      NaN mins      NA mins      NA mins      -Inf mins      NA      NA      0 mins      0.133
6 Send for Credit Collection      Inf mins      NA mins      NaN mins      NA mins      NA mins      -Inf mins      NA      NA      0 mins      0.0975
7 Insert Date Appeal to Prefecture      Inf mins      NA mins      NaN mins      NA mins      NA mins      -Inf mins      NA      NA      0 mins      0.00668
8 Send Appeal to Prefecture      Inf mins      NA mins      NaN mins      NA mins      NA mins      -Inf mins      NA      NA      0 mins      0.00654
9 Receive Result Appeal from Prefecture      Inf mins      NA mins      NaN mins      NA mins      NA mins      -Inf mins      NA      NA      0 mins      0.00158
10 Notify Result Appeal to Offender      Inf mins      NA mins      NaN mins      NA mins      NA mins      -Inf mins      NA      NA      0 mins      0.00156
11 Appeal to Judge      Inf mins      NA mins      NaN mins      NA mins      NA mins      -Inf mins      NA      NA      0 mins      0.000547
> processing_time(traffic_fines, level="log", units="days")
# A tibble: 1 x 8
  min      q1      median      mean      q3      max      st_dev      iqr
  <drtn>      <drtn>      <drtn>      <drtn>      <drtn>      <drtn>      <dbl>      <drtn>
1 0 days 0 days 0 days 0 days 0 days 0 days 0 0 days

```

No es normal que un análisis de minería de procesos nos arroje *Inf*, *NaN* o puros ceros. Lo que sucede aquí es que el paquete *edeaR* calcula el tiempo de procesamiento basándose en dos marcas de tiempo: **start** (inicio) y **complete** (final). Pero en este caso en los registros solo están registrados el momento exacto en que ocurrió la acción (un solo timestamp), no cuánto duró la ejecución de la misma. Al no haber una diferencia de tiempo entre el inicio y el fin de una actividad (porque ocurren en el mismo instante registrado), el resultado es 0 o error.

Para estos casos, lo más útil no es ver cuánto dura la actividad en sí (*processing_time*), sino cuánto tiempo pasa **entre** actividades o cuánto dura el caso completo desde que inicia hasta que cierra.

Así como otros cambios que hemos tenido que adecuar.

Comando que había	Comando que usamos	Razón
<code>Processing_time</code>	<code>Thoroughput_time</code>	Las multas no tienen duración de tarea, sino tiempo de espera
<code>Blood test</code>	<code>Payment</code>	<i>Payment</i> corresponde a una etapa real de una multa de tráfico. <i>Blood Test</i> era una etapa del set anterior
<code>units = 'mins'</code>	<code>units = 'days'</code>	Nadie paga una multa en minutos. Estos datos usan escalas de días.

En un proceso de multas, lo interesante no es cuánto tardan en imprimir el papel, sino **cuánto tiempo pasa entre un paso y otro** (por ejemplo, cuánto tardan desde que crean la multa hasta que el ciudadano la paga).

Los datos son más consistentes, serán mostrados en el apartado de cálculo.

Pero podemos obtener un resumen rápido con una fila de resumen que en este caso incluye un filtro:

En lugar de medir cuánto "tarda" el pago (que es 0), mide cuánto tarda toda la multa (log) para los casos que incluyeron un pago.

```

> traffic_fines %>% filter_activity_presence("Payment") %>% throughput_time(level="log", units="days")
# A tibble: 1 x 8
  min      q1      median      mean      q3      max      st_dev      iqr
  <drtn>      <drtn>      <drtn>      <drtn>      <drtn>      <drtn>      <dbl>      <drtn>
1 0 days 2 days 6 days 107.179 days 138.5 days 1260 days 189. 136.5 days

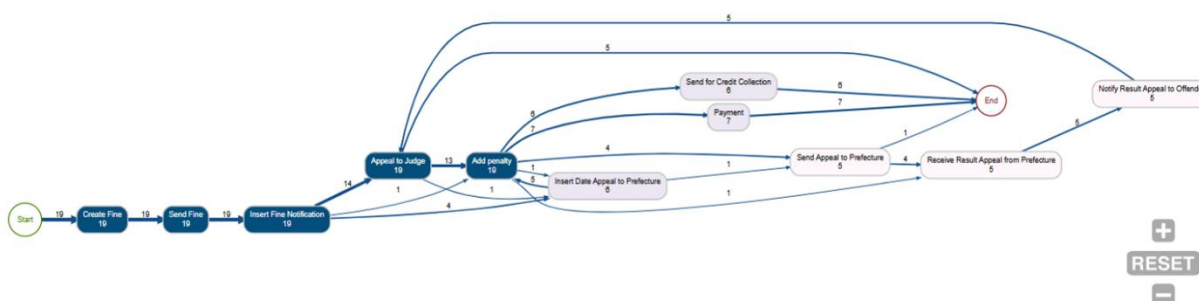
```

En base a un filtro (que ahora es distinto) podemos obtener un mapa de proceso. Hemos elegido hacer nuestro mapa de proceso que representa los distintos procesos que, según los registros, terminaron en juicio.

```
multas_juicio <- filter_activity_presence(traffic_fines, "Appeal to Judge")
```

Esto quiere decir que, de nuestra tabla de registros de datos que enlistan casos y configuraciones de eventos sobre las multas de tráfico, estamos eligiendo sólo las que incluyeron una *Apelación a juicio* y ahora haremos un mapa de procesos de esas configuraciones, como se ve a continuación:

```
# 3. Genera el mapa de ese subset
multas_juicio %>% process_map(type = frequency("absolute"))
```



En la sección de análisis detallaremos más sobre el mapa obtenido.

Cálculos y resultados

Cálculos del mapa de procesos en R

Cálculos de actividades y frecuencias

Los cálculos de esta actividad son bastante reducidos, pero podemos partir del primer paso que involucra cálculos por parte de R, las frecuencias de cada actividad.

```
> activities(traffic_fines)
# A tibble: 11 x 3
  activity          absolute_frequency relative_frequency
  <fct>          <int>          <dbl>
1 Create Fine      10000          0.288
2 Send Fine        6570          0.189
3 Payment          4910          0.141
4 Add penalty      4635          0.133
5 Insert Fine Notification 4635          0.133
6 Send for Credit Collection 3387          0.0975
7 Insert Date Appeal to Prefecture 232          0.00668
8 Send Appeal to Prefecture 227          0.00654
9 Receive Result Appeal from Prefecture 55          0.00158
10 Notify Result Appeal to Offender 54          0.00156
11 Appeal to Judge  19          0.000547
```

Aquí podemos observar que una multa de tráfico, que es un proceso compuesto por 11 etapas,

tiene a su vez, etapas que ocurren con mayor frecuencia que otras. Esto es lógico, porque hay procesos incompletos o con pasos que son opcionales. Por ejemplo, no necesariamente todas las multas tienen que terminar en una apelación a juicio.

De hecho, *Appeal to Judge* es la actividad del proceso con la frecuencia más baja. De todos los casos registrados de multas, sólo 19 fueron a juicio. Esto es especialmente llamativo para mí y por eso elegí hacer el mapa de procesos (que está más adelante en el documento) con únicamente procesos que involucran esa actividad.

```
> throughput_time(traffic_fines, level="log", units="days")
# A tibble: 1 x 8
  min    q1  median  mean    q3   max  st_dev iqr
  <drtn> <drtn> <drtn> <drtn> <drtn> <drtn> <dbl> <drtn>
1 0 days 7 days 125 days 296.1351 days 596 days 1956 days 313. 589 days
```

Después se exploran los datos de los tiempos de proceso de las multas. En unidades de días, porque como mencionamos antes, es una escala de tiempo apropiada para las multas de tráfico dado a que usualmente la gente no paga una multa de tráfico en menos de un día.

Y de este cálculo de promedio obtenemos una conclusión importante:

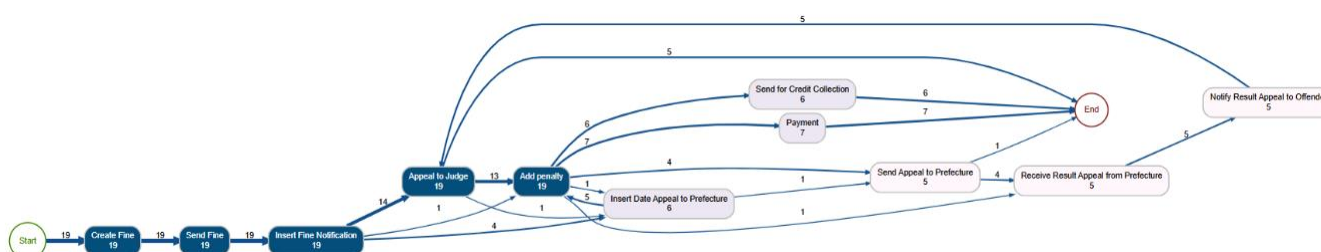
En promedio, una multa tarda **296 días** en cerrarse (mean), pero hay casos extremos que duran hasta **1956 días** (max). ¡Casi 5 años!

Cálculos del mapa de procesos

Finalmente para el mapa de procesos, se han seleccionado y almacenado en un subset todos los procesos que involucran la actividad *Appeal To Judge*, es decir, casos de multas que terminaron apelando a juicio.

```
multas_juicio <- filter_activity_presence(traffic_fines, "Appeal to Judge")
```

Y de dicho Subset hemos construido un mapa de proceso con ayuda de la librería *process_map*.



Lo que tenemos aquí es un mapa de **frecuencias absolutas** de una muestra muy específica (exactamente 19 multas de tráfico). Los números dentro de las cajas indican cuántas veces ocurrió esa actividad, y los números en las flechas nos dicen cuántos casos viajaron por esa ruta específica.

Aunque los procesos expresan una diversidad operacional, podemos identificar dos patrones y una conclusión a partir de ella

Un tronco común

Si observamos el lado izquierdo del mapa (el inicio), notamos que todos los 19 casos inician exactamente igual y sin desviaciones:

Start -> Create Fine (19) -> Send Fine (19) -> Insert Fine Notification (19).

El proceso administrativo inicial es perfectamente estandarizado y eficiente. No hay cuellos de botella ni saltos extraños al momento de crear y notificar la multa.

Las apelaciones

El proceso se rompe justo después de que el ciudadano es notificado. En un mundo ideal, la flecha iría directo a pagar, pero observamos algo distinto.:

De las 19 notificaciones, la gran mayoría (**14 casos**) se desvían hacia **Appeal to Judge** (Apelar al Juez). Esperaríamos que las 19 se fueran a apelación porque seleccionamos justamente un dataset con casos que incluyeran de manera forzosa sólo casos que tuvieran apelación. Y es así, pero aquí hay una distinción importante, estos 14 casos se fueron **directamente a apelar**. El resto de los casos que hicieron apelaciones, pasaron por otros procesos antes de apelar a juicio.

Hay una altísima tasa de inconformidad. En lugar de resolver el problema rápido, el proceso se atasca en trámites legales y burocráticos.

Hallazgo principal: Nadie paga a tiempo

Este es el dato más interesante que encontré: **nadie paga la multa original**.

Si buscamos el nodo **Payment** (Pago), podemos observar que tiene 7 casos.

Sin embargo, la única flecha que llega a **Payment** viene de **Add penalty** (Añadir recargo).

Esto quiere decir que los 7 ciudadanos que finalmente pagaron, lo hicieron *después* de que se les aplicó una penalización económica. Cero personas pagaron de forma voluntaria al recibir la notificación inicial. Al menos en los casos que involucran un juicio.

Las Resoluciones

El proceso desemboca en el círculo rojo (**End**) de cuatro formas distintas, lo que demuestra mucha variabilidad en la conclusión de los casos:

- **7 casos** terminan tras pagar con recargo.
- **6 casos** nunca pagan y se envían a cobranza externa (**Send for Credit Collection**).
- **5 casos** se cierran directamente por orden del juez (**Appeal to Judge -> End**).
- **1 caso** se cierra tras notificar el resultado de una apelación a la prefectura.

Síntesis del análisis del ejercicio con edeaR

Observamos entonces que la cadena de valor en el proceso de multas de tráfico analizado presenta una **fricción crítica** inmediatamente después de la notificación inicial. Lo que encontramos es un sistema con una tendencia general hacia la **judicialización y el pago tardío**; la ausencia total de pagos voluntarios antes de la aplicación de recargos (Add penalty) sugiere que el

incentivo de pago temprano es ineficaz o que los ciudadanos prefieren agotar las instancias de apelación ante el juez antes de cumplir con la obligación financiera. Esto convierte a lo que debería ser un flujo administrativo lineal en un proceso 'espagueti' con altos tiempos de ciclo (promedio de **296 días**) y una carga operativa significativa para el sistema judicial.

Conclusión de la práctica

Esta práctica resultó ser una de las más ilustrativas del laboratorio. Aunque la práctica anterior introdujo el concepto de mapas de procesos, siento que esta fue la que realmente nos permitió meternos al tema para entender cómo se construyen estos modelos desde la raíz. La primera parte de la práctica requería traducir nuestro mapa a otros siete tipos de notaciones formales (como Redes de Petri o BPMN); aunque lamento no haber profundizado en esa comparativa (Porque no hice esta parte), el enfoque en eventdataR compensó el aprendizaje técnico.

El desafío principal fue romper la intuición inicial al enfrentarme a la representación de los tiempos. El primer obstáculo (encontrar ceros en el tiempo de procesamiento) tuvo mucho sentido una vez que comprendí la naturaleza de las **actividades instantáneas** o eventos de un solo *timestamp*. Entender que en estos casos la clave no está en la duración de la actividad, sino en el **Throughput Time** (el tiempo que pasa *entre* actividades), fue una lección fundamental de minería de procesos: la estructura del dato dicta el tipo de análisis posible.

Además, aprendí que la limpieza y el filtrado de subsets no son solo pasos técnicos, sino herramientas de diagnóstico, como pudimos experimentar en la *Práctica 1* también. Al aislar comportamientos específicos (como los casos que llegan a juicio), el 'ruido' de los datos desaparece y permite identificar cuellos de botella que de otro modo estarían ocultos en el promedio general del log.

Referencias

- [1]. W. M. P. van der Aalst, *Process Mining: Data Science in Action*. Springer, 2016.
- [2]. T. H. Davenport, *Process Innovation: Reengineering Work through Information Technology*. Harvard Business School Press, 1993.
- [3]. K. Alves de Medeiros, A. J. M. M. Weijters, and W. M. P. van der Aalst, "Genetic Process Mining," *Applications and Theory of Petri Nets*, 2005.
- [4]. W. van der Aalst, "Process Mining and Data Science: What's the Connection?" *IEEE Intelligent Systems*, vol. 31, no. 5, pp. 92–95, 2016.
- [5]. T. Murata, "Petri Nets: Properties, Analysis and Applications," *Proceedings of the IEEE*, vol. 77, no. 4, pp. 541–580, 1989.
- [6]. Fluxicon, "Disco Process Mining Tool – User Guide," Fluxicon, 2023.